

BIG DATA ANALYTICS WITH LAB

Experiments 1-6 - Updated Practical Record for Windows 11

Software / Platform

Component	Version / Setup
Operating System	Windows 11
Linux Environment	WSL 2 + Ubuntu
Java	OpenJDK 17
Hadoop	Apache Hadoop 3.5.0
Python	Python 3
Container Platform	Docker Desktop
Cassandra	Apache Cassandra 5.0.9

This document replaces the older Linux/Hadoop 2.x-oriented procedures with the verified Windows 11 + WSL2 practical workflow used for these six experiments.

EXPERIMENT NO. 1

Install Hadoop and configure Hadoop Distributed File System (HDFS) in pseudo-distributed mode.

AIM

To install Apache Hadoop 3.5.0 on Windows 11 using WSL2 and Ubuntu, configure HDFS and YARN in pseudo-distributed mode, and verify that the Hadoop daemons run successfully.

DESCRIPTION

Hadoop is a distributed data-processing platform. In this setup, Windows 11 hosts WSL2, Ubuntu provides the Linux environment, Java 17 provides the runtime, and Hadoop 3.5.0 provides HDFS and YARN. The pseudo-distributed configuration runs the Hadoop daemons on a single machine while using HDFS and YARN services.

ALGORITHM

- Enable and verify WSL2 with Ubuntu.
- Install OpenJDK 17 and verify the Java version.
- Download and extract Hadoop 3.5.0 and place it under /opt/hadoop.
- Set JAVA_HOME, HADOOP_HOME and related Hadoop environment variables.
- Configure hadoop-env.sh with the Java installation path.
- Install and configure SSH so Hadoop start/stop scripts can access localhost.
- Configure core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml.
- Create NameNode and DataNode storage directories.
- Format the NameNode once.
- Start HDFS and YARN and verify NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager with jps.
- Create a test file in HDFS and verify it can be listed, uploaded and displayed.

DETAILED STEPS

1. Verify WSL2 from Windows PowerShell

```
wsl --version
wsl -l -v
```

2. Install Java 17 in Ubuntu

```
sudo apt update
sudo apt install openjdk-17-jdk -y
java -version
```

3. Download Hadoop

```
cd ~
wget https://downloads.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
ls -lh hadoop-3.5.0.tar.gz
```

4. Extract and install Hadoop

```
tar -xzf hadoop-3.5.0.tar.gz
sudo mv hadoop-3.5.0 /opt/hadoop
ls /opt/hadoop
```

5. Add environment variables to ~/.bashrc

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/hadoop
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
```

```
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
source ~/.bashrc
hadoop version
```

6. Configure hadoop-env.sh

```
nano /opt/hadoop/etc/hadoop/hadoop-env.sh
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

7. Configure localhost SSH

```
sudo apt install openssh-server openssh-client -y
sudo service ssh start
ssh-keygen -t ed25519 -N ""
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost "echo SSH_OK"
```

EXPERIMENT 1 - XML CONFIGURATION

8. Configure core-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

9. Configure hdfs-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/wwwka/hadoopdata/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/wwwka/hadoopdata/datanode</value>
  </property>
</configuration>
```

10. Create storage directories

```
mkdir -p ~/hadoopdata/namenode
mkdir -p ~/hadoopdata/datanode
```

11. Configure mapred-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>${HADOOP_MAPRED_HOME}/share/hadoop/mapreduce/*:${HADOOP_MAPRED_HOME}/share/hadoop/mapreduce/lib/*</value>
  </property>
</configuration>
```

12. Configure yarn-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
    <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOO
  </property>
</configuration>
```

13. Format and start Hadoop

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps
```

EXPECTED JPS OUTPUT

NameNode

DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps

14. HDFS verification

```
hdfs dfs -mkdir -p /lab/input
echo "Hadoop 3.5.0 is working on Windows 11" > test.txt
hdfs dfs -put test.txt /lab/input/
hdfs dfs -ls /lab/input
hdfs dfs -cat /lab/input/test.txt
```

RESULT

Hadoop 3.5.0 was installed successfully in WSL2/Ubuntu on Windows 11. HDFS and YARN were configured and started in pseudo-distributed mode, and the required Hadoop daemons were verified using jps.

EXPERIMENT NO. 2

Demonstrate basic HDFS commands for file creation, upload, download, replication, and deletion operations.

AIM

To perform file and directory management operations in HDFS, including creation, upload, display, download, copy, rename, replication checking, and deletion.

DESCRIPTION

HDFS provides command-line utilities that resemble common file-management commands. The experiment demonstrates how local files are moved into HDFS, retrieved from HDFS, copied and renamed within HDFS, checked for replication, and deleted.

ALGORITHM

- Create a local working directory and sample text file.
- Create the required directory in HDFS.
- Upload the local file to HDFS.
- List and display the HDFS file.
- Download the file back to local storage.
- Copy and rename a file within HDFS.
- Check the replication factor.
- Delete files and directories.

PROGRAM / COMMANDS

```
mkdir -p ~/bigdata/exp2
cd ~/bigdata/exp2

nano example.txt

hdfs dfs -mkdir -p /exp2/input
hdfs dfs -put example.txt /exp2/input/
hdfs dfs -ls /exp2/input
hdfs dfs -cat /exp2/input/example.txt

mkdir -p ~/bigdata/exp2/download
hdfs dfs -get /exp2/input/example.txt ~/bigdata/exp2/download/

hdfs dfs -cp /exp2/input/example.txt /exp2/example_copy.txt
hdfs dfs -mv /exp2/example_copy.txt /exp2/renamed_example.txt

hdfs dfs -stat "%n  replication=%r" /exp2/input/example.txt

hdfs dfs -rm /exp2/renamed_example.txt
hdfs dfs -rm /exp2/input/example.txt
hdfs dfs -rm -r /exp2
```

INPUT

Hadoop is a distributed framework.
HDFS stores large files reliably.
Hadoop supports big data processing.

OUTPUT

```
example.txt  replication=1
Deleted /exp2/renamed_example.txt
Deleted /exp2/input/example.txt
Deleted /exp2
```

RESULT

The required HDFS file-management operations were successfully performed using Hadoop 3.5.0.

EXPERIMENT NO. 3

Develop a MapReduce program to perform Word Count analysis on a text dataset.

AIM

To develop a Python MapReduce Word Count program using Hadoop Streaming and count the frequency of each word in a text dataset.

DESCRIPTION

The Mapper reads each input line, extracts words, and emits each word with the value 1. Hadoop performs the Shuffle and Sort phase. The Reducer receives grouped words and sums their counts to produce the final word-frequency table.

ALGORITHM

- Read each line from standard input in the Mapper.
- Normalize the line to lowercase and extract words.
- Emit each word with value 1 as a tab-separated key-value pair.
- Hadoop groups and sorts identical keys.
- Read grouped key-value pairs in the Reducer.
- Add counts for identical words and print the final count.

PROGRAM - mapper.py

```
#!/usr/bin/env python3
import re
import sys

for line in sys.stdin:
    line = line.strip().lower()
    words = re.findall(r"[a-zA-Z0-9'"]+", line)

    for word in words:
        print(f"{word}\t1")
```

PROGRAM - reducer.py

```
#!/usr/bin/env python3
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    try:
        word, count = line.split("\t", 1)
        count = int(count)
    except ValueError:
        continue

    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")

            current_word = word
            current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

INPUT - input.txt

```
Hadoop is a Big Data framework
Hadoop uses MapReduce
Python can be used with Hadoop Streaming
MapReduce processes Big Data efficiently
```

LOCAL TEST

```
cat input.txt | python3 mapper.py | sort | python3 reducer.py
```

HADOOP EXECUTION

```
hdfs dfs -mkdir -p /exp3/input
hdfs dfs -put input.txt /exp3/input/

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar \
-input /exp3/input \
-output /exp3/output \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
```

OUTPUT

```
a          1
be         1
big        2
can        1
data       2
efficiently 1
framework  1
hadoop     3
is         1
mapreduce  2
processes  1
python     1
streaming  1
used       1
uses       1
with       1
```

OUTPUT VERIFICATION

```
hdfs dfs -cat /exp3/output/part-*
```

RESULT

The Python Word Count MapReduce program was successfully executed using Hadoop Streaming, and the expected word frequencies were obtained.

EXPERIMENT NO. 4

Implement Hadoop Streaming using Python scripts for simple data processing tasks.

AIM

To implement Hadoop Streaming using Python Mapper and Reducer scripts to process student records and calculate the department-wise average marks.

DESCRIPTION

The Mapper reads comma-separated student records and emits the department as the key and marks as the value. Hadoop performs Shuffle and Sort to group records by department. The Reducer calculates the average marks for each department.

ALGORITHM

- Read each student record from standard input.
- Validate the three CSV fields: name, department and marks.
- Emit department and marks as a tab-separated pair.
- Hadoop groups all records having the same department.
- The Reducer adds the marks and counts the students for each department.
- Compute and print the department average to two decimal places.

PROGRAM - mapper.py

```
#!/usr/bin/env python3
import csv
import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    try:
        row = next(csv.reader([line]))

        if len(row) != 3:
            continue

        name, department, marks = row
        department = department.strip()
        marks = int(marks.strip())

        print(f"{department}\t{marks}")

    except (ValueError, csv.Error):
        continue
```

PROGRAM - reducer.py

```
#!/usr/bin/env python3
import sys

current_department = None
total_marks = 0
student_count = 0

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    try:
        department, marks = line.split("\t", 1)
        marks = int(marks)
    except ValueError:
```

```

        continue

    if department == current_department:
        total_marks += marks
        student_count += 1
    else:
        if current_department is not None:
            average = total_marks / student_count
            print(f"{current_department}\tAverage = {average:.2f}")

        current_department = department
        total_marks = marks
        student_count = 1

if current_department is not None:
    average = total_marks / student_count
    print(f"{current_department}\tAverage = {average:.2f}")

```

INPUT - students.txt

```

Arun,CSE,82
Priya,AIML,91
Karthik,CSE,75
Divya,AIML,88
Rahul,IT,79
Meena,CSE,95
Siva,IT,85
Anitha,AIML,94

```

LOCAL TEST

```
cat students.txt | python3 mapper.py | sort | python3 reducer.py
```

HADOOP EXECUTION

```

hdfs dfs -mkdir -p /exp4/input
hdfs dfs -put students.txt /exp4/input/

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar \
-input /exp4/input \
-output /exp4/output \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py

```

OUTPUT

```

AIML    Average = 91.00
CSE     Average = 84.00
IT      Average = 82.00

```

RESULT

The Hadoop Streaming application was successfully implemented and the department-wise average marks were calculated correctly.

EXPERIMENT NO. 5

Perform data storage and retrieval operations using Cassandra Query Language (CQL).

AIM

To perform data storage and retrieval operations using Apache Cassandra CQL by creating a keyspace and table, inserting student records, executing retrieval queries, updating a record, and deleting a record.

DESCRIPTION

For the Windows 11 lab, Cassandra is run in Docker Desktop using the WSL2 integration. CQLSH is opened inside the Cassandra container. The experiment demonstrates keyspace creation, table creation, INSERT, SELECT, UPDATE and DELETE operations.

ALGORITHM

- Start Docker Desktop with WSL2 integration enabled for Ubuntu.
- Pull the Cassandra 5.0.9 image.
- Create the cassandra-lab container and expose port 9042.
- Open CQLSH inside the container.
- Create and use the studentdb keyspace.
- Create the student table.
- Insert five student records.
- Retrieve all records and selected records.
- Update Priya's marks from 91 to 94.
- Delete Rahul's record and verify the remaining rows.

DOCKER SETUP

```
docker --version
docker run --rm hello-world

docker pull cassandra:5.0.9
docker run --name cassandra-lab -p 9042:9042 -d cassandra:5.0.9
docker ps

docker exec cassandra-lab nodetool status
docker exec -it cassandra-lab cqlsh
```

CQL PROGRAM

```
CREATE KEYSPACE IF NOT EXISTS studentdb
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};

USE studentdb;

CREATE TABLE IF NOT EXISTS student (
    student_id INT PRIMARY KEY,
    name TEXT,
    department TEXT,
    year INT,
    marks INT
);

INSERT INTO student VALUES (101, 'Arun', 'CSE', 3, 82);
INSERT INTO student VALUES (102, 'Priya', 'AIML', 3, 91);
INSERT INTO student VALUES (103, 'Rahul', 'IT', 3, 79);
INSERT INTO student VALUES (104, 'Divya', 'AIML', 3, 88);
INSERT INTO student VALUES (105, 'Meena', 'CSE', 3, 95);

SELECT * FROM student;
```

```

SELECT * FROM student
WHERE student_id = 102;

SELECT name, department, marks
FROM student;

SELECT * FROM student
WHERE department = 'AIML'
ALLOW FILTERING;

UPDATE student
SET marks = 94
WHERE student_id = 102;

SELECT * FROM student
WHERE student_id = 102;

DELETE FROM student
WHERE student_id = 103;

SELECT * FROM student;

```

OUTPUT

student_id	department	marks	name	year
105	CSE	95	Meena	3
104	AIML	88	Divya	3
102	AIML	94	Priya	3
101	CSE	82	Arun	3

(4 rows)

STOP / START CONTAINER

```

docker stop cassandra-lab
docker start cassandra-lab

```

RESULT

Cassandra storage and retrieval operations were successfully performed using CQL. The keyspace and table were created, records were inserted and retrieved, one record was updated, and one record was deleted successfully.

EXPERIMENT NO. 6

Create and manipulate Key-Value and Document-oriented data models using NoSQL databases.

AIM

To create and manipulate Key-Value and Document-Oriented data models using Python by performing Create, Read, Update and Delete (CRUD) operations.

DESCRIPTION

The Key-Value model is represented with a Python dictionary in which a product ID identifies a product name. The Document-Oriented model is represented with a list of dictionaries in which each dictionary is a JSON-like product document containing multiple fields.

ALGORITHM

- Create a Python dictionary for the Key-Value model.
- Display all products and retrieve one product by key.
- Update an existing value, add a new key-value pair, and delete an existing key.
- Create a list of dictionaries for the Document-Oriented model.
- Display all documents and filter products by category.
- Update a product price, append a new document, and delete a document.
- Display the final collections.

PROGRAM - exp6_nosql.py

```
#!/usr/bin/env python3

print("=" * 60)
print("KEY-VALUE DATA MODEL")
print("=" * 60)

products = {
    "P101": "Laptop",
    "P102": "Smartphone",
    "P103": "Headphones"
}

print("\nAvailable Products:")
for product_id, product_name in products.items():
    print(f"{product_id} : {product_name}")

print("\nRetrieve Product P102:")
print(products.get("P102", "Product not found"))

products["P102"] = "5G Smartphone"

print("\nAfter Updating P102:")
for product_id, product_name in products.items():
    print(f"{product_id} : {product_name}")

products["P104"] = "Tablet"

print("\nAfter Adding P104:")
for product_id, product_name in products.items():
    print(f"{product_id} : {product_name}")

del products["P103"]

print("\nAfter Deleting P103:")
for product_id, product_name in products.items():
    print(f"{product_id} : {product_name}")

print("\n" + "=" * 60)
print("DOCUMENT-ORIENTED DATA MODEL")
print("=" * 60)
```

```

documents = [
    {
        "ProductID": "P101",
        "ProductName": "Laptop",
        "Category": "Electronics",
        "Price": 65000,
        "Stock": 20
    },
    {
        "ProductID": "P102",
        "ProductName": "Smartphone",
        "Category": "Electronics",
        "Price": 28000,
        "Stock": 35
    },
    {
        "ProductID": "P103",
        "ProductName": "Office Chair",
        "Category": "Furniture",
        "Price": 7500,
        "Stock": 15
    }
]

print("\nAll Products:")
for product in documents:
    print(product)

print("\nElectronics Products:")
for product in documents:
    if product["Category"] == "Electronics":
        print(product)

for product in documents:
    if product["ProductID"] == "P102":
        product["Price"] = 26000

print("\nAfter Updating P102 Price:")
for product in documents:
    print(product)

documents.append({
    "ProductID": "P104",
    "ProductName": "Tablet",
    "Category": "Electronics",
    "Price": 22000,
    "Stock": 18
})

print("\nAfter Adding P104:")
for product in documents:
    print(product)

documents = [
    product for product in documents
    if product["ProductID"] != "P103"
]

print("\nAfter Deleting P103:")
for product in documents:
    print(product)

print("\n" + "=" * 60)
print("CRUD OPERATIONS COMPLETED SUCCESSFULLY")
print("=" * 60)

```

EXECUTION

```

mkdir -p ~/bigdata/exp6
cd ~/bigdata/exp6
nano exp6_nosql.py
python3 exp6_nosql.py

```

OUTPUT

KEY-VALUE DATA MODEL

```

Available Products:
P101 : Laptop
P102 : Smartphone
P103 : Headphones

```

Retrieve Product P102:
Smartphone

After Updating P102:
P101 : Laptop
P102 : 5G Smartphone
P103 : Headphones

After Adding P104:
P101 : Laptop
P102 : 5G Smartphone
P103 : Headphones
P104 : Tablet

After Deleting P103:
P101 : Laptop
P102 : 5G Smartphone
P104 : Tablet

DOCUMENT-ORIENTED DATA MODEL

After Updating P102 Price:
P102 -> Price 26000

After Adding P104:
P104 -> Tablet -> Price 22000

After Deleting P103:
P101 -> Laptop
P102 -> Smartphone -> Price 26000
P104 -> Tablet -> Price 22000

CRUD OPERATIONS COMPLETED SUCCESSFULLY

RESULT

The Key-Value and Document-Oriented data models were successfully created and manipulated using Python. CRUD operations were demonstrated successfully.

FINAL PRACTICAL VERIFICATION

Status of all six experiments

Experiment	Status	Verified Work
1	COMPLETED	Hadoop 3.5.0, HDFS, YARN, SSH, pseudo-distributed cluster
2	COMPLETED	HDFS create, put, get, cat, cp, mv, replication, rm
3	COMPLETED	Python Word Count through Hadoop Streaming
4	COMPLETED	Department-wise average through Hadoop Streaming
5	COMPLETED	Cassandra keyspace/table + INSERT/SELECT/UPDATE/DELETE
6	COMPLETED	Python Key-Value and Document CRUD

Record note

The documentation above is based on the updated practical workflow actually verified during the lab sessions. Experiment 1 uses pseudo-distributed Hadoop on one Windows 11 machine through WSL2; Experiment 5 uses Cassandra 5.0.9 through Docker Desktop.